Initial Results: Two-Tier Control Plane for 6G Slicing

Dataset: network slicing dataset (Farreras et al. 2024, Zenodo 10610616). Prototype-first, 1-month scope.

1. Pipeline steps

1. Load a sample: per-slice offered load (eMBB / URLLC / mMTC) + global delay/loss.

2. Tier 1 (fast ML): linear forecaster predicts next aggregate load; numpy MLP predicts delay; p_violation from delay vs budget.

3. Analytic capacity model converts per-slice load -> required replicas.

4. Payload = per-slice KPIs + ML signals + current replicas -> Tier 2 LLM.

5. LLM proposes per-slice replica counts (MCP typed tool, or YAML manifest).

6. Verification gate checks schema + policy, then applies.

7. Hindsight oracle (offline, greedy P95) scores the decision vs future load.

2. Models

Tier 1 (fast ML): linear forecaster (window 5) + numpy MLP regressor (load -> delay), no sklearn. Baseline: predict-then-threshold.

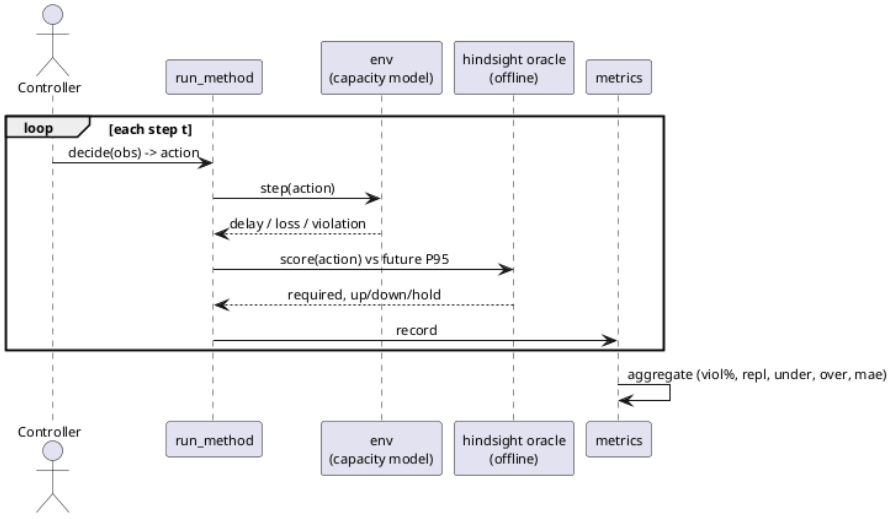
Tier 2 (LLM, Ollama cloud over localhost, OpenAI-compatible): glm-5.3-flash:cloud, glm-5.3:cloud, mistral-large-4:cloud, deepseek-v4-pro:cloud. Temperature 0, JSON output.

Actuation: MCP typed tools (estimate_capacity, scale_replicas, set_resources, add_nf_instance, apply_k8s_manifest).

Ground truth: analytic capacity model + greedy P95 hindsight oracle (prescient: looks at future load; offline only).

3. Method and controllers (detailed)

Harness: every controller is the same decide(obs) -> action. run_method feeds the action to the env and the oracle; metrics() aggregates.



Environment / capacity model

- state per slice = {load, replicas, delay, loss}
- action = {slice: replicas}, clamped to [1, 5]

- effective_capacity = replicas × cap_per_replica
- rho (utilization) = load / effective_capacity
- delay = base_delay / (1 − rho) when rho < 1, else max_delay
- loss = (load − capacity) / load when overloaded, else 0
- violation = delay > delay_budget OR loss > loss_budget
- NFConfig per slice: URLLC tight delay (4× base), eMBB loose (30× base), mMTC strict loss (0.5%)

Hindsight oracle (offline)

- at step t, look at future per-slice load over horizon H=5 (offset d=1)
- required = ceil(P95 / (cap × target_util))
- label up/down/hold vs the chosen action
- prescient — used only for scoring, never shown to the system

threshold

- reactive — acts only after a violation
- always one step late

```
if delay > budget: replicas = required(load)
elif delay < 0.5*budget and replicas > min: replicas -= 1
else: hold
```

hpa

- ideal controller — always matches the capacity model (mae 0)
- residual violation = the max_replicas=5 cap at peak load

```
replicas = required(load) = ceil(load / (cap * target_util))
```

predict-then-threshold

- linear forecaster predicts next aggregate load
- numpy MLP (load -> delay) predicts delay, then a threshold decides
- degenerates to HPA: the MLP over-predicts delay, so it almost always scales up

```
pred_delay = mlp(forecaster(history))
if pred_delay > budget: scale every slice to required(load)
elif pred_delay < 0.5*budget: scale down
else: hold
```

llm-stub

- deterministic, no-network stand-in for the real LLM; same payload in, fixed rule out
- numbers are not hand-set: the switch-case picks the action, the env + oracle produce the metrics
- shares the required_replicas capacity model with the oracle, so it is a near-optimal upper-bound reference — intentionally close to the solution, not a competitor to beat

```
if p_violation > 0.5 and required > current: replicas = required
elif p_violation < 0.1 and current > min: replicas -= 1
else: hold
```

llm-real

- glm-5.3-flash:cloud via Ollama (OpenAI-compatible, localhost, no token), temperature 0
- receives the per-slice payload + ML signals, outputs JSON {actions, reason}
- ~6 s per decision
- conservative: over-provisions (over=0.96) to keep violation at 7.7%

4. Metrics used and why

Metric	Why it matters
SLA violation %	Outcome: how often delay/loss exceeds the slice budget (the cost the operator cares about).
mean replicas	Resource cost: over-provisioning wastes compute.
under-scaling	Missed scale-ups (replicas below oracle requirement) - the key small-model failure.
over-scaling	Unnecessary scale-ups (replicas above oracle requirement).
scale-up recall / precision	Decision quality: does the controller scale up when it should (recall) and only then (precision).
hold correctness	Does the controller leave well-enough alone when no change is needed.
MAE vs intended	Fidelity: distance from the ideal capacity-model decision (Mode T).
latency / tokens	Tier-2 cost: wall-clock and token cost of the LLM decision (RQ3).

5. Benchmark results (per-slice closed loop)

Scenario: **ramp**

method	viol%	repl	under	over	upRec	upPrec	holdOk	mae
threshold	24.67	2.50	1.11	0.11	0.12	0.32	0.46	0.49
hpa	15.67	2.63	0.95	0.08	0.10	0.21	0.57	0.00
predict-then-threshold	15.67	2.63	0.95	0.08	0.10	0.21	0.57	0.00
llm-stub	1.00	3.73	0.16	0.38	0.02	0.09	0.97	1.10
llm-flash	7.67	3.93	0.53	0.96	0.16	0.17	0.54	1.48
llm-glm5.3	1.00	4.86	0.06	1.42	0.20	0.11	0.96	2.23
llm-mistral	5.00	4.41	0.38	1.28	0.08	0.09	0.85	1.81
llm-deepseek	2.33	4.65	0.21	1.36	0.24	0.23	0.91	2.05

Scenario: **steady**

method	viol%	repl	under	over	upRec	upPrec	holdOk	mae
threshold	27.50	2.58	1.28	0.22	0.12	0.30	0.17	0.59
hpa	16.67	2.63	1.13	0.12	0.09	0.19	0.40	0.00
predict-then-threshold	16.67	2.63	1.13	0.12	0.09	0.19	0.40	0.00
llm-stub	0.83	4.28	0.12	0.76	0.11	0.22	0.96	1.65
llm-flash	5.83	4.11	0.50	0.97	0.19	0.20	0.38	1.53
llm-glm5.3	1.25	4.87	0.11	1.34	0.45	0.36	0.92	2.24
llm-mistral	6.25	4.19	0.56	1.10	0.08	0.09	0.52	1.56
llm-deepseek	3.75	4.50	0.32	1.18	0.31	0.28	0.68	1.87

Scenario: **spike**

method	viol%	repl	under	over	upRec	upPrec	holdOk	mae
threshold	20.00	2.15	1.11	0.19	0.11	0.36	0.52	0.44
hpa	14.58	2.24	0.95	0.12	0.15	0.33	0.62	0.00
predict-then-threshold	15.00	2.23	0.96	0.12	0.14	0.31	0.62	0.01
llm-stub	1.67	4.06	0.22	1.21	0.09	0.33	0.95	1.82
llm-flash	5.83	3.95	0.40	1.28	0.12	0.12	0.66	1.87
llm-glm5.3	2.08	4.64	0.13	1.71	0.15	0.12	0.91	2.42
llm-mistral	4.58	4.50	0.29	1.73	0.13	0.13	0.88	2.33
llm-deepseek	2.92	4.58	0.23	1.74	0.24	0.20	0.90	2.38

6. RQ5 — does the estimate_capacity tool help?

Same model (glm-5.3-flash:cloud), same 20 states, with vs without tool-calling. Tool mode routes calls through the MCP server (estimate_capacity + scale_replicas).

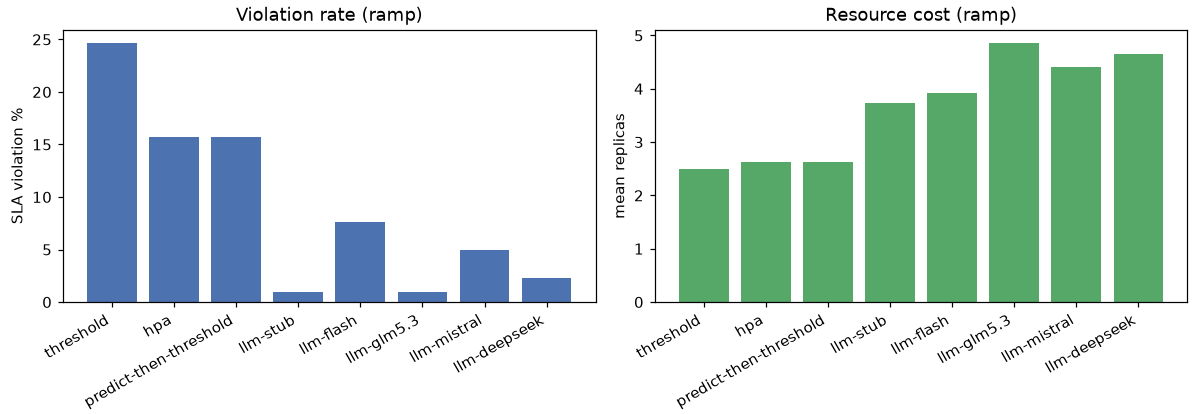
mode	viol%	repl	mae	under	over
no_tool	6.67	3.47	1.18	0.45	0.37
tool	11.67	2.55	0.20	1.10	0.10

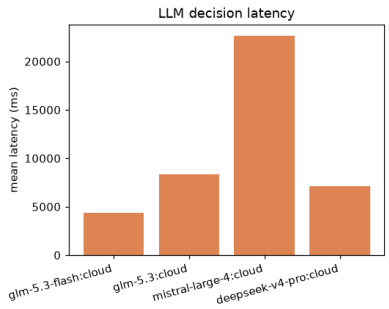
Finding: the tool cuts MAE 1.18 -> 0.20 and cost 3.47 -> 2.55 replicas (the LLM defers to the model), but raises violation 6.7% -> 11.7% because the model's 0.7 target utilization is tighter than the LLM's own headroom. The tool makes the LLM exact, not safer.

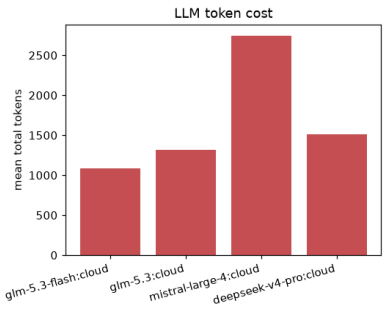
7. LLM calls (4 per model)

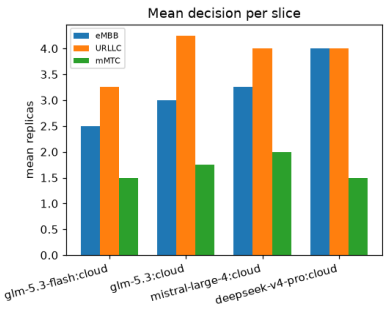
model	call	eMBB	URLLC	mMTC	lat(ms)	tokens	reason
glm-5.3-flash:cloud	1	1	1	3	6451.90	1551	mMTC violates delay (0.00426
glm-5.3-flash:cloud	2	3	4	1	5029.90	956	eMBB and URLLC both violate
glm-5.3-flash:cloud	3	3	5	1	2883.70	872	eMBB and URLLC both violate
glm-5.3-flash:cloud	4	3	3	1	3205.90	965	eMBB and URLLC severely viol
glm-5.3:cloud	1	1	2	4	12030.60	1655	mMTC severely violates delay
glm-5.3:cloud	2	3	5	1	5734.20	1028	URLLC loss (0.386 vs 0.005)
glm-5.3:cloud	3	4	5	1	5032.10	957	eMBB and URLLC grossly viola
glm-5.3:cloud	4	4	5	1	10626.40	1619	eMBB and URLLC grossly viola
mistral-large-4:cloud	1	1	1	5	23304.80	2053	mMTC severely violates delay
mistral-large-4:cloud	2	3	5	1	18538.60	3023	URLLC severe KPI violations
mistral-large-4:cloud	3	4	5	1	23838.40	3332	eMBB and URLLC exhibit sever
mistral-large-4:cloud	4	5	5	1	25010.90	2577	eMBB/URLLC show critical los
deepseek-v4-pro:cloud	1	1	1	3	15174.50	2658	mMTC KPI violation; scale up
deepseek-v4-pro:cloud	2	5	5	1	1728.20	707	eMBB and URLLC KPI violation
deepseek-v4-pro:cloud	3	5	5	1	4597.20	1188	eMBB and URLLC KPI violation
deepseek-v4-pro:cloud	4	5	5	1	6893.10	1501	eMBB and URLLC have severe d

8. Comparison graphs




LLM decision latency
mean latency (ms)
0
5000
10000
15000
20000
glm-5.3-flash:cloud
glm-5.3:cloud
mistral-large-4:cloud
deepseek-v4-pro:cloud


LLM token cost
mean total tokens
0
500
1000
1500
2000
2500
glm-5.3-flash:cloud
glm-5.3:cloud
mistral-large-4:cloud
deepseek-v4-pro:cloud


Mean decision per slice
eMBB
URLLC
mMTC
mean replicas
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
glm-5.3-flash:cloud
glm-5.3:cloud
mistral-large-4:cloud
deepseek-v4-pro:cloud